Planejamento ServerRest

1. Informações Gerais

Este documento define o Plano de Testes para a atividade da semana 10, seguindo uma abordagem simplificada e focada nos requisitos essenciais da norma ISO/IEC/IEEE 29119-3 facilitada.

1.1. Identificação do Projeto/Sistema

- **Nome do Projeto/Sistema:** Planejamento de Teste - API ServerRest
- **Data de Criação do Plano:** 02 de fev. de 2026
- **Elaborado por:** Igor Herculano Silva
- **API testada:**

ServeRest
compassuol.serverest.dev

1.2. Objetivos e Escopo dos Testes

Este plano foca em reconhecer e validar as regras de negocio a partir dos dados documentos no Swagger.

- **Objetivos dos Testes:**
  - Verificar **endpoints**
  - Verificar o **fluxo de valor**
  - Verificar funcionamento dos **tokens**
  - Treinar Técnicas de testes exploratórios, testes funcionais, engenharia de qualidade e planejamento de testes..

1.3. Escopo (Por modulo)

Usuário	Login	Produto	Carrinho	AUTH
Cadastro (POST)	Geração de **Token**	Cadastro (POST)	Cadastro (POST)	Cadastro (POST)
Busca,listagem (GET)	-	Busca, listagem (GET)	Busca, listagem (GET)	Busca,listagem (GET)
Edição (PUT)	-	Edição (PUT)	Exclusão (DELETE)	Edição (PUT)
Exclusão (DELETE)	-	Exclusão (DELETE)		Exclusão (DELETE)

1.4. Fora do escopo

- Testes de **Carga** e **Stress**
- **Interface**
- **Infra** e **Banco de Dados**
- Testes de usabilidade da **Documentação** ou da **Lógica de Negócios**

2. Abordagem de Teste

A abordagem de teste será manual e exploratória, focada na validação de contratos, códigos de status HTTP e lógica de negócios, utilizando ferramentas de requisição (**Postman**) e a documentação **Swagger.**.

2.1. Estratégia de Teste

A execução dos testes exploratórios será gerenciada através da metodologia de **Gestão de Testes Baseada em Sessão** (SBTM). As sessões serão focadas em validar os endpoints, manipulando o JSON de entrada (payload) para verificar a robustez da API e a integridade dos dados.

2.2. Tipos de Teste

- **Testes de Contrato:** Validação da estrutura do JSON de resposta e Tipagem de dados.
- **Testes Funcionais:** Verificação das regras de negócio (CRUD) e Status Codes (200, 201, 400, 401).
- **Testes de Segurança Básica:** Validação de acesso a rotas protegidas sem token ou com token inválido.

3. Ambiente de Teste

3.1. Necessidades de Hardware e Software

Tipo	Especificação
Servidor de Aplicação	Ambiente Externo de Produção
Estações de Teste	Windows 11, Postman
Documentação	Swagger

3.2. Dados de Teste

Os dados do teste serão de uma conta real criada para fins de teste.

4. Critérios de entrada

4.1. Critérios de entrada

O ciclo de testes só poderá iniciar quando:

- A API estiver acessível publicamente na URL `compassuol.serverest.dev/`
- As *Collections* do Postman estiverem criadas com as variáveis de ambiente configuradas (`base_url`).
- O ambiente no Jira/QAlity estiver organizado para criação de ciclos e bugs.
- A documentação (Swagger) estiver disponível para consulta.
- O documento de Plano de Testes estiver finalizado e entregue na plataforma de ensino (Learning).

4.2. Critérios de saída

- 100% dos Cenários de Teste planejados (Seção 6) forem executados.
- Todos os defeitos Críticos e Altos encontrados estiverem devidamente reportados no Jira com evidências.
- A taxa de realização dos testes de "Caminho Feliz" (Reconhecimento da API) for 100%.

5. Gerenciamento de Teste

5.1. Gerenciamento de Casos de Teste

O gerenciamento do ciclo de vida dos testes será realizado na ferramenta **Jira**, utilizando o plugin **QAlity** para estruturação e execução.

- **Repositório:** Os cenários de teste serão criados no QAlity e vinculados às Histórias de Usuário para garantir rastreabilidade.
- **Execução:** A execução técnica ocorre no Postman, e o status (Pass/Fail) é atualizado no "Test Cycle" do QAlity.

5.2. Gerenciamento de Não Conformidades (Defeitos)

Os defeitos encontrados serão catalogados no **Jira** como *Bugs*, seguindo rigorosamente o template abaixo:

Template de Report de Bug:

6. Cenários de teste funcionais

ID	Pilar (VADER)	Cenário de Teste	Objetivo Técnico	Resultado Esperado
CT-01	Verbs	Enviar PUT na rota /login	Validar se a API bloqueia verbos não suportados.	Status 405 Method Not Allowed.
CT-02	Verbs	Enviar GET na rota /login	Verificar se credenciais trafegam via URL.	Status 405 ou 400.
CT-03	Authorization	Tentar cadastrar Produto sem enviar Header Authorization	Validar barreira de entrada.	**Status 401** Unauthorized; Msg: "Token de acesso ausente".
CT-04	Authorization	Tentar cadastrar Produto com Token de usuário comum	Validar RBAC.	**Status 403** Forbidden; Msg: "Rota exclusiva para administradores".
CT-05	Authorization	Manipular Token e tentar POST /produtos	Testar integridade da assinatura do Token.	**Status 401** Unauthorized.
CT-06	Data	Cadastrar usuário com e-mail inválido	Validar sanitização e formato de dados.	**Status 400** Bad Request.
CT-07	Data	Cadastrar produto com preço negativo	Validar regras de negócio e integridade financeira.	**Status 400** Bad Request.

CT-08	Data	Cadastrar usuário enviando campo administrador como booleano true	Validar tipagem estrita.	**Status 400** (Falha de validação de schema).
CT-09	Data	Enviar Payload vazio {} no POST /usuarios	Validar obrigatoriedade de campos.	**Status 400** Bad Request; Listar campos obrigatórios faltantes.
CT-10	Errors	Tentar excluir usuário com ID inexistente	Verificar tratamento de recurso não encontrado.	**Status 200** (Mensagem: "Nenhum registro excluído") ou **400**.
CT-11	Errors	Enviar JSON malformado	Verificar se a API trata parse error sem expor stack trace.	**Status 400** Bad Request.
CT-12	Errors	Tentar cadastrar e-mail já existente	Verificar mensagem de erro de duplicidade.	**Status 400** Msg: "Este email já está sendo usado".
CT-13	Responsiveness	Medir tempo de resposta do login	Garantir UX aceitável.	**Status 200** Tempo < (**Número a determinar**)
CT-14	Responsiveness	Listar produtos (GET /produtos)	Verificar performance de consulta ao banco.	**Status 200** Tempo < (**Número a determinar**)
CT-15	Authorization	(Novo) Admin cadastrar novo Produto (Fluxo Happy Path)	Validar privilégio positivo: Admin deve conseguir criar recursos.	**Status 201** Created; Retornar ID do produto.
CT-16	Authorization	(Novo) Admin excluir Produto existente	Validar privilégio positivo: Admin deve conseguir gerenciar ciclo de vida.	**Status 200 OK**; Msg: "Registro excluído com sucesso".
CT-17	Authorization	(Novo) Usuário comum tentar excluir Produto	Validar proteção do método DELETE contra usuários sem permissão.	**Status 403 Forbidden**; Msg: "Rota exclusiva para administradores".
CT-18	Authorization	(Novo) Tentar elevação de privilégio via "Parameter Tampering" (enviar admin: true no Body)	Validar segurança contra injeção de parâmetros (Mass Assignment).	**Status 403 Forbidden** (A API deve ignorar o campo injetado).
CT-19	Authorization	(Novo) Tentar elevação de privilégio via "Header Injection" (x-role: admin)	Validar que a API confia apenas no Token JWT e ignora headers falsos.	**Status 403 Forbidden**.

7. Riscos

Risco (Falha Provável)	Probabilidade	Impacto	Exemplo possíveis
Falso Positivo em Autenticação	**Média**	**Crítico**	A API permitir acesso a rotas de Admin usando token de usuário comum.
Inconsistência de Dados (Lixo no Banco)	**Média**	**Médio**	A API aceitar cadastro de produtos com preço negativo ou nomes vazios.
Persistência Volátil	**Alta**	**Baixo**	O servidor reiniciar e apagar os usuários criados no teste anterior.

8. Mapeamento de Endpoints

Módulo	Método	Endpoint	Objetivo Principal
Login	**POST**	/login	Autenticar usuário e gerar Token.
Usuários	**GET**	/usuarios	Listar todos os cadastros.
	POST	/usuarios	Cadastrar novo usuário.

	DELETE	/usuarios/{_id}	Excluir usuário.
Produtos	GET	/produtos	Listar produtos disponíveis.
	POST	/produtos	Cadastrar novo produto (Requer Token).
Carrinhos	POST	/carrinhos	Criar carrinho com produtos.
	DELETE	/carrinhos/concluir-compra	Finalizar compra e limpar carrinho.
	DELETE	/carrinhos/cancelar-compra	Cancelar compra e limpar carrinho.

9. Exemplos de requisição

1. Login

Endpoint: `/login`

POST (Autenticação)

JSON

```
{
  "email": "fulano.qa@teste.com.br",
  "password": "teste"
}
```

---

2. Usuários

Endpoint: `/usuarios`

POST (Cadastro de Usuário)

JSON

```
{
  "nome": "Igor QA Teste",
  "email": "igor.qa.teste1@teste.com.br",
  "password": "teste",
  "administrador": "true"
}
```

PUT (Edição de Usuário) *URL:* `/usuarios/{_id}`

JSON

```
{
  "nome": "Igor QA Editado",
  "email": "igor.qa.teste1@teste.com.br",
  "password": "nova_senha_123",
  "administrador": "true"
}
```

---

3. Produtos

Endpoint: `/produtos` *Requisito: Header* `Authorization: Bearer <seu_token>`

POST (Cadastro de Produto)

JSON

```
{
  "nome": "Notebook Gamer",
  "preco": 12000,
  "descricao": "Notebook para jogos de alta performance",
  "quantidade": 10
}
```

PUT (Edição de Produto) *URL:* `/produtos/{_id}`

JSON

```
{
  "nome": "Notebook Gamer
  V2",
  "preco": 11500,
  "descricao": "Versão atualizada com mais RAM",
  "quantidade": 15
}
```

---

4. Carrinhos

Endpoint: `/carrinhos`

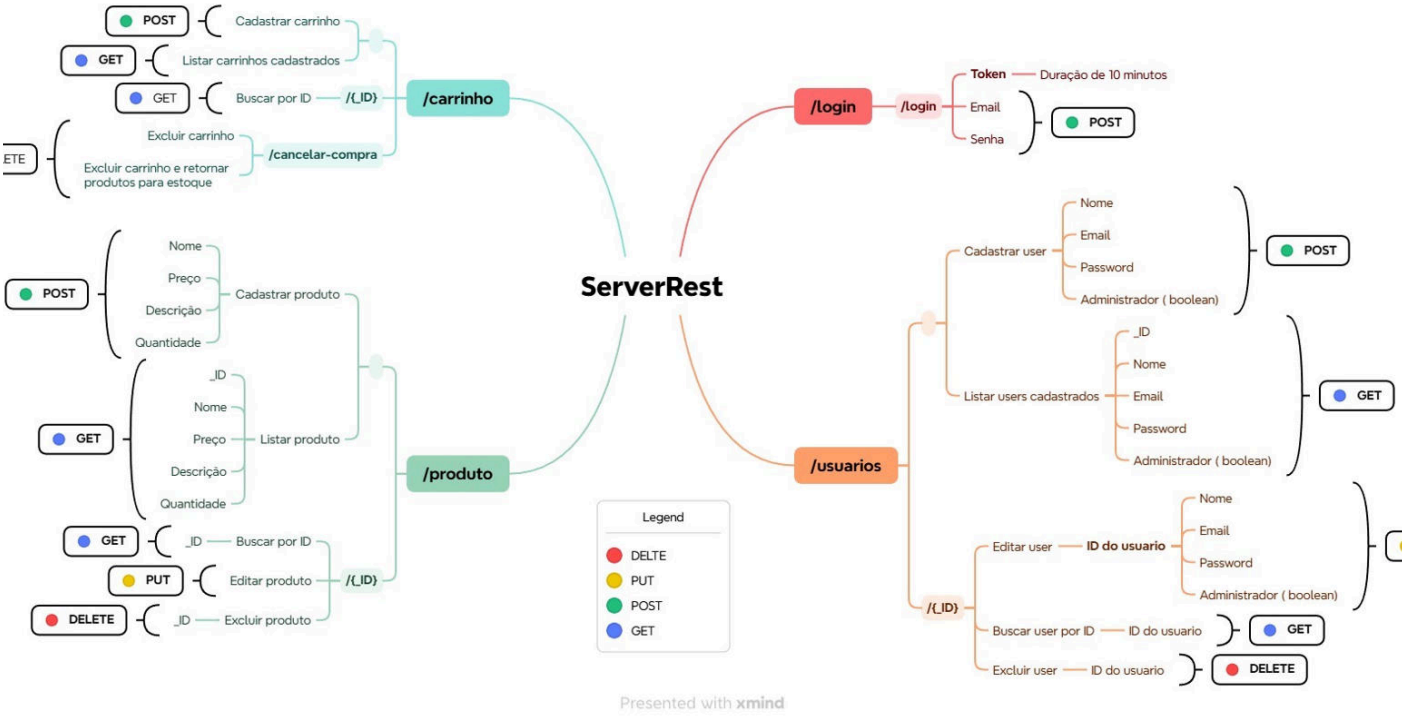
POST (Criar Carrinho)

JSON

```
{
  "produtos": [
    {
      "idProduto": "BeeJh5lz3k6kSIzA",
      "quantidade": 1
    },
    {
      "idProduto": "K6leHdftCeOjj8BJ",
      "quantidade": 2
    }
  ]
}
```

---

10. Mapa Mental



11. Session Sheets

Session Sheet 1

ID da Sessão:	#01 - Exploração de Usuário e Login
Testador:	Igor Herculano Silva
Data:	11/02/2026
Duração (Timebox):	30 minutos **(Planejado)** / 45 minutos **(Real)**

Objetivo (Charter):	Explorar o endpoint de autenticação (/login), validando segurança de verbos HTTP, estrutura do token e tempos de resposta.
Ambiente:	Windows 11 / Postman / API ServerRest (Produção)

Detalhes da Execução (Session Log)

Resumo da Abordagem: A sessão foi focada em testes de API utilizando o Postman no ambiente *ServerRest*. Foram executados testes funcionais, de segurança básica e de performance no endpoint de autenticação.

O que foi testado:

1. **Validação de "Caminho Feliz" (Happy Path):**
   - Envio de requisição `POST` para `/login` com e-mail e senha cadastrados previamente.
   - Validação do recebimento de Status Code `200 OK`.
   - Confirmação de que a resposta contém a chave `authorization` (Bearer Token).
2. **Segurança de Verbos HTTP:**
   - Tentativa de acesso ao endpoint `/login` utilizando métodos incorretos: `GET`, `PUT`, `DELETE` e `PATCH`.
   - Objetivo: Garantir que a API rejeita métodos não permitidos (esperado 405 ou 404).
3. **Análise da Estrutura do Token (JWT):**
   - Cópia do token gerado e decodificaçã para validar se o header, payload e assinatura respeitam a estrutura de três partes separadas por pontos.
   - Verificação se o token não está vazio ou `null`.
4. **Monitoramento de Performance:**
   - Execução de 5 chamadas consecutivas para verificar a latência média.
   - Validação se o tempo de resposta se manteve dentro de um padrão aceitável para a produção (abaixo de 700ms-1s).

ID	Cenário de Teste	Resultado	Observação
CT-001	Realizar Login com credenciais válidas	✅**Aprovado**	Retornou token e status 200.
CT-002	Validar segurança de verbos HTTP (GET/PUT em /login)	✅**Aprovado**	API bloqueou métodos indevidos corretamente.
CT-003	Verificar estrutura do formato JWT	✅**Aprovado**	Token segue padrão header.payload.signature.
CT-004	Validar tempo de resposta do endpoint	✅**Aprovado**	Média de resposta estável (~250ms).
CT-005	Verificar obrigatoriedade de Body na requisição	✅**Aprovado**	Requisição sem body retornou erro tratado (400).
CT-009	Verificar campos obrigatorios	❌ **Falhou**	API aceitou os campos Nome e senha

Bugs e Problemas Encontrados

⊗

BUG - Violação de Tipagem - Severidade Média: A API aceitou a entrada com **Status 201 Created**, criando um usuário com nome e senha "invisíveis".

Session Sheet 2

ID da Sessão:	#02 - Exploração de Produto e carrinho
Testador:	Igor Herculano Silva
Data:	11/02/2026
Duração (Timebox):	45 minutos **(Planejado)** / 100 minutos **(Real)**
Objetivo (Charter):	Explorar o endpoint de autenticação Produto e carrinho
Ambiente:	Windows 11 / Postman / API ServerRest (Produção)

Detalhes da Execução (Session Log)

Resumo da Abordagem: A sessão focou na transição entre ações de Administrador (Gestão de Produtos) e Usuário Final (Fluxo de Compra), com forte ênfase em testes de borda e validação de contratos de API.

O que foi testado:

1. Módulo: Produtos (Perfil Admin)

- **Cenário V (Verbos):** Tentativa de uso de verbos não suportados (`PATCH` , `PUT` na raiz). *Comportamento Correto (405).*
- **Cenário D (Dados/Tipagem):**
  - Envio de Strings em campos numéricos (`quantidade: "10"`). **Falha Encontrada (Bug #01).**
  - Validação de campos obrigatórios. *Validado.*
- **Cenário E (Erros/Duplicidade):**
  - Cadastro de produtos com nomes idênticos. **Falha Encontrada (Bug #02).**
  - Exclusão de IDs inexistentes. *Tratado corretamente.*

2. Módulo: Carrinhos (Perfil Usuário)

- **Cenário D (Dados/Integridade):**
  - Tentativa de adicionar ID de produto inexistente. *Bloqueado (400).*
  - Tentativa de comprar quantidade maior que o estoque. *Bloqueado (400 - Estoque insuficiente).*
- **Cenário E (Erros/Concorrência):**
  - Tentativa de criar dois carrinhos simultâneos para o mesmo usuário. *Bloqueado (400 - Não permitido mais de 1 carrinho).*
- **Cenário Automação (Fluxo Autocontido):**
  - Implementação de script `Pre-request` para criação dinâmica de massa de dados (evitando erro de "ID em branco").
  - Implementação de `Teardown` para limpeza automática do banco após o teste.

Módulo	CT - ID	Descrição do Teste	Status	Observação
Produtos	CT-01	Bloqueio de Verbo Não Suportado (PATCH/PUT)	✅ **Aprovado**	API retornou 405 Method Not Allowed corretamente.
Produtos	CT-08	Validação de Tipagem (Campo quantidade como String)	❌ **Falhou**	Bug-01: Aceitou string "10" e converteu para int (Retornou 201).
Produtos	CT-12	Validação de Unicidade (Nome Duplicado)	✅ **Aprovado**	Bug-02: Permitiu cadastrar produtos com nomes idênticos.

Produtos	CT-10	Tratamento de Exclusão de ID Inexistente	✅ **Aprovado**	API tratou a requisição sem erros críticos (200/404).
Produtos	CT-13	Performance de Listagem (Tempo de Resposta)	✅ **Aprovado**	Resposta dentro do limite aceitável.
Carrinhos	CT-01	Bloqueio de Verbo Não Suportado (PUT na raiz)	✅ **Aprovado**	API retornou 405 Method Not Allowed.
Carrinhos	CT-10	Integridade de Dados (ID de Produto Inexistente)	✅ **Aprovado**	API bloqueou a compra e avisou "Produto não encontrado".
Carrinhos	CT-08	Regra de Negócio (Quantidade > Estoque)	✅ **Aprovado**	API bloqueou a compra e avisou sobre estoque insuficiente.
Carrinhos	CT-12	Concorrência/Unicidade (Múltiplos Carrinhos)	✅ **Aprovado**	API impediu a criação de um 2º carrinho para o mesmo usuário.
Carrinhos	CT-13	Performance de Listagem	✅ **Aprovado**	Resposta rápida.

Bugs e Problemas Encontrados

❌ **BUG-01 - Violação de Tipagem - Severidade Baixa**: API aceita o envio do campo `quantidade` como **String** (ex: `"10"`) e realiza a correção automática para número (Status 201), violando o contrato estrito de tipos inteiros. .

Session Sheet 3

ID da Sessão:	#01 - Exploração de Usuário e Login
Testador:	Igor Herculano Silva
Data:	12/02/2026
Duração (Timebox):	30 minutos **(Planejado)** / 25 minutos **(Real)**
Objetivo (Charter):	Validar os mecanismos de **Autorização (RBAC)** da API. Verificar se os privilégios de Administrador e Usuário Comum são respeitados, garantindo que rotas protegidas bloqueiem acessos não autorizados.
Ambiente:	Windows 11 / Postman / API ServerRest (Produção)

Detalhes da Execução (Session Log)

Resumo da Abordagem: A sessão focou na segurança ofensiva e validação de perfis de acesso (Role-Based Access Control). Foram executados fluxos completos para **Administradores** (que devem ter permissão total) e **Usuários Comuns** (que devem ter permissão restrita), além de tentativas de burlar a segurança via manipulação de parâmetros.

O que foi testado:

1. **Fluxo de Administrador (Privilégio Total):**

a. **Cenário A (Auth):** Cadastro de usuário com flag `administrador: "true"`. Login realizado com sucesso.
b. **Cenário V (Verbos/Ação):** Criação de produto (`POST /produtos`) com token de Admin. **Resultado: Sucesso (201 Created).**
c. **Cenário E (Erros/Limpeza):** Exclusão do produto criado para garantir que Admins podem gerenciar o ciclo de vida completo. **Resultado: Sucesso (200 OK).**

2. **Fluxo de Usuário Comum (Privilégio Restrito):**
   a. **Cenário A (Auth):** Cadastro de usuário com flag `administrador: "false"`. Login realizado e token capturado.
   b. **Cenário de Bloqueio:** Tentativa de criar um produto utilizando o token de usuário comum. **Resultado: Bloqueado (403 Forbidden).** A API retornou a mensagem correta: *"Rota exclusiva para administradores"*.
   c. **Cenário de Segurança Ofensiva:** Tentativa de "Parameter Injection" enviando o campo `"administrador": "true"` no corpo da requisição de criação de produto, mesmo estando logado como usuário comum. **Resultado: Bloqueado (403 Forbidden).** A API ignorou a tentativa de elevação de privilégio via Body.

Módulo	CT - ID	Descrição do Teste	Status	Observação
Auth	CT-04	**Permissão de Admin** Verificar se Admin consegue criar recursos protegidos.	✅ **Aprovado**	API permitiu a criação (Status 201) corretamente.
Auth	CT-05	**Bloqueio de Usuário Comum** Verificar se Usuário Comum é impedido de criar produtos.	✅ **Aprovado**	API bloqueou o acesso (Status 403 Forbidden).
Auth	CT-15	**Segurança de Parâmetros** Tentativa de injetar admin: true no Body da requisição.	✅ **Aprovado**	API ignorou o parâmetro injetado e manteve o bloqueio (403).
Auth	CT-16	**Segurança de Headers** Tentativa de enviar headers falsos (x-role: admin).	✅ **Aprovado**	API ignorou os headers e validou apenas o Token JWT original.
Auth	CT-13	**Performance de Login** Validar tempo de resposta na autenticação.	✅ **Aprovado**	Login realizado em < 200ms.

Bugs e Problemas Encontrados

Nenhum bug de segurança foi encontrado nesta sessão: A API demonstrou robustez no controle de acesso.

11. Bugs Encontrados

Type	Key	Resumo	Responsável	Prioridade	Status
	SCRUM-15	Erro de escrita no Swagger		Low	EM ANÁLISE
	SCRUM-14	Falta de Validação em Campos Obrigatórios (No...		High	EM ANÁLISE
	SCRUM-12	Validação de Tipagem QTD em String invés de INT	IH igor H.	Medium	EM ANÁLISE

Synced just now • 3 items